Kristiania

Artificial Neural Network

Instructor: Mohib Ullah

PGR216: Applications of AI and Cyber Security

09 March 2026

Content

- Motivation
- Perceptron
- Activation function
- XOR problem
- Multi-layer perceptron
- Backpropagation
- Overfitting & underfitting
- Use of MLP in classification and Regression
- Bias-Variance Tradoff

Motivation: Why Neural Networks?

Limitations of Linear Models

- Decision function:

$$y = \mathbf{w}^T\mathbf{x} + b$$

- This represents a **hyperplane** in feature space.

Key limitations:

- Can only model **linear relationships** between input and output
- Fail when:
  - Classes are not linearly separable
  - The relationship is highly non-linear (e.g., XOR problem)
- Require heavy **manual feature engineering** to capture non-linearity:
  - Polynomial features
  - Domain-specific transformations

Linear models are simple, fast, and interpretable, but too restrictive for many real-world problems.

Complex Decision Boundaries

Real data often requires:

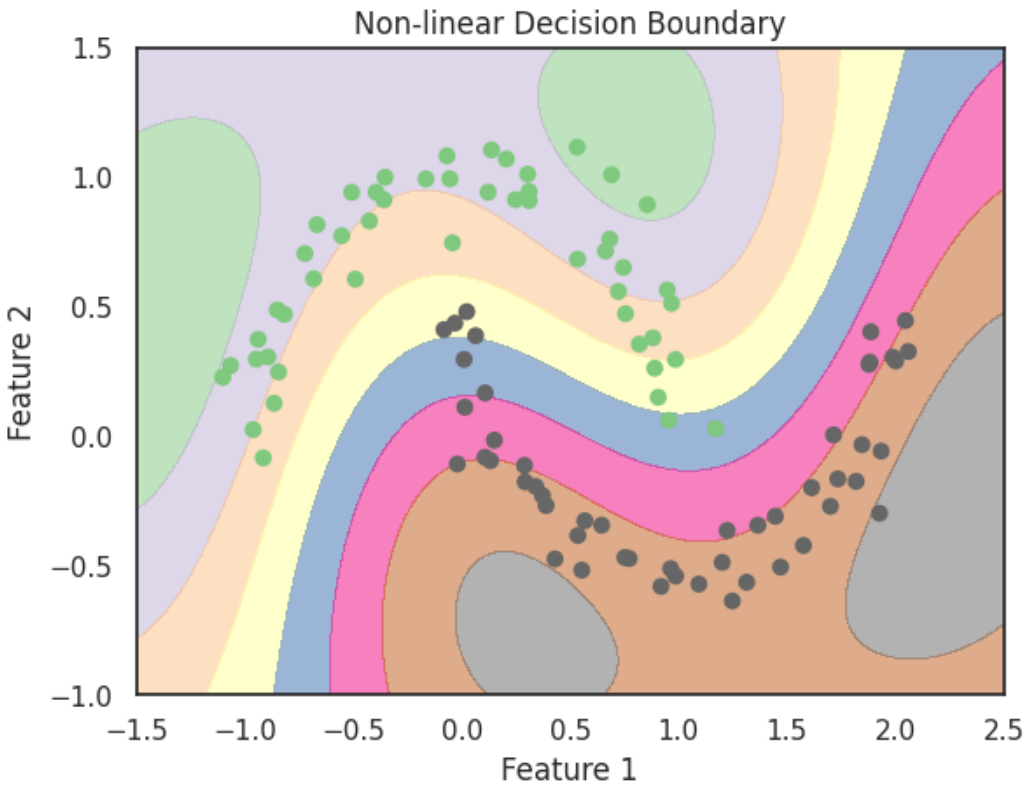
- Curved boundaries
- Multiple disconnected regions
- Hierarchical patterns

Neural networks provide:

- Non-linear activation functions (ReLU, sigmoid, tanh)
- Multiple layers models **composition of functions**
- This allows them to approximate:
- $f(x) = f_3\left(f_2\left(f_1(x)\right)\right)$

Results:

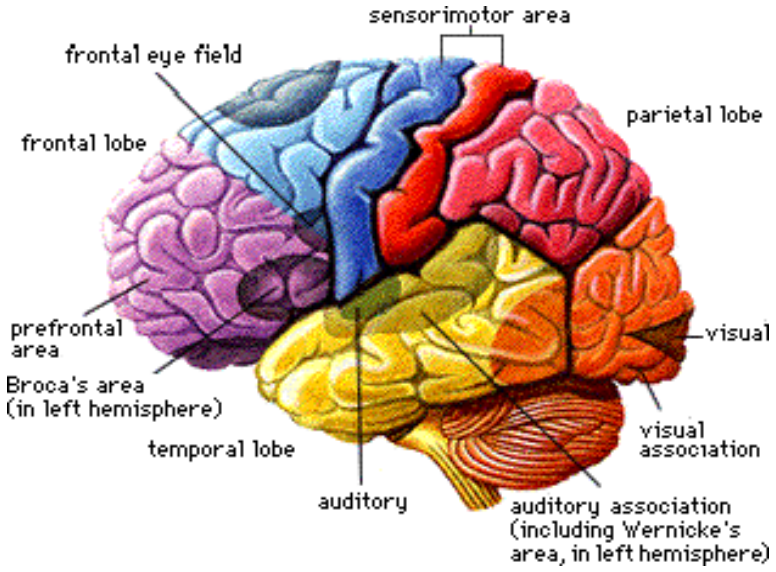
- Arbitrarily complex decision boundaries
- Universal Approximation Theorem:
  - A sufficiently large MLP can approximate any continuous function

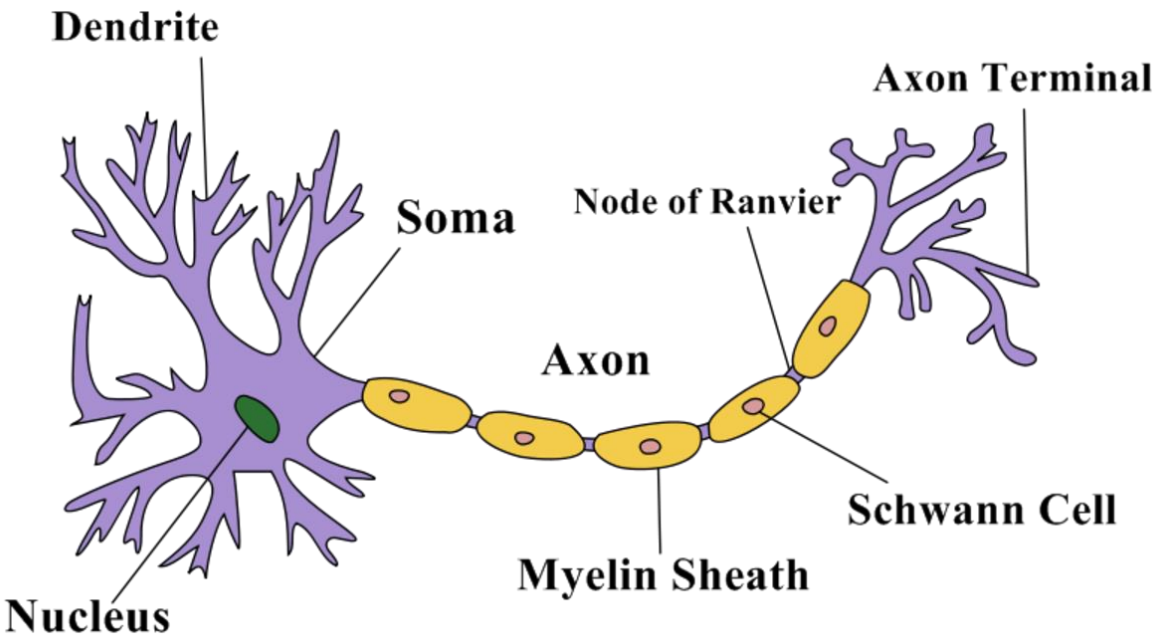


https://iq.opengenus.org/decision-boundary/

Artificial Neural Networks

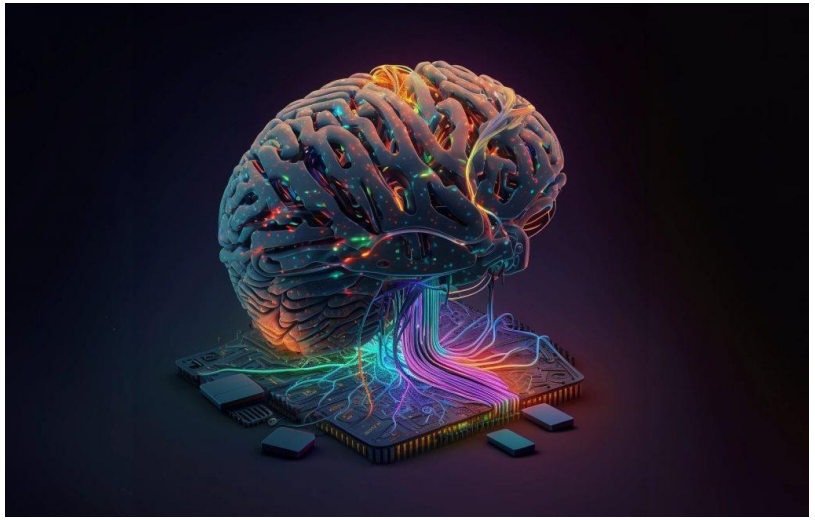
- The interest in the neural networks is motivated by the capability of the human brain to solve in a short time very different and complex problems, despite the "data" transmission velocity is very low (order of milliseconds).





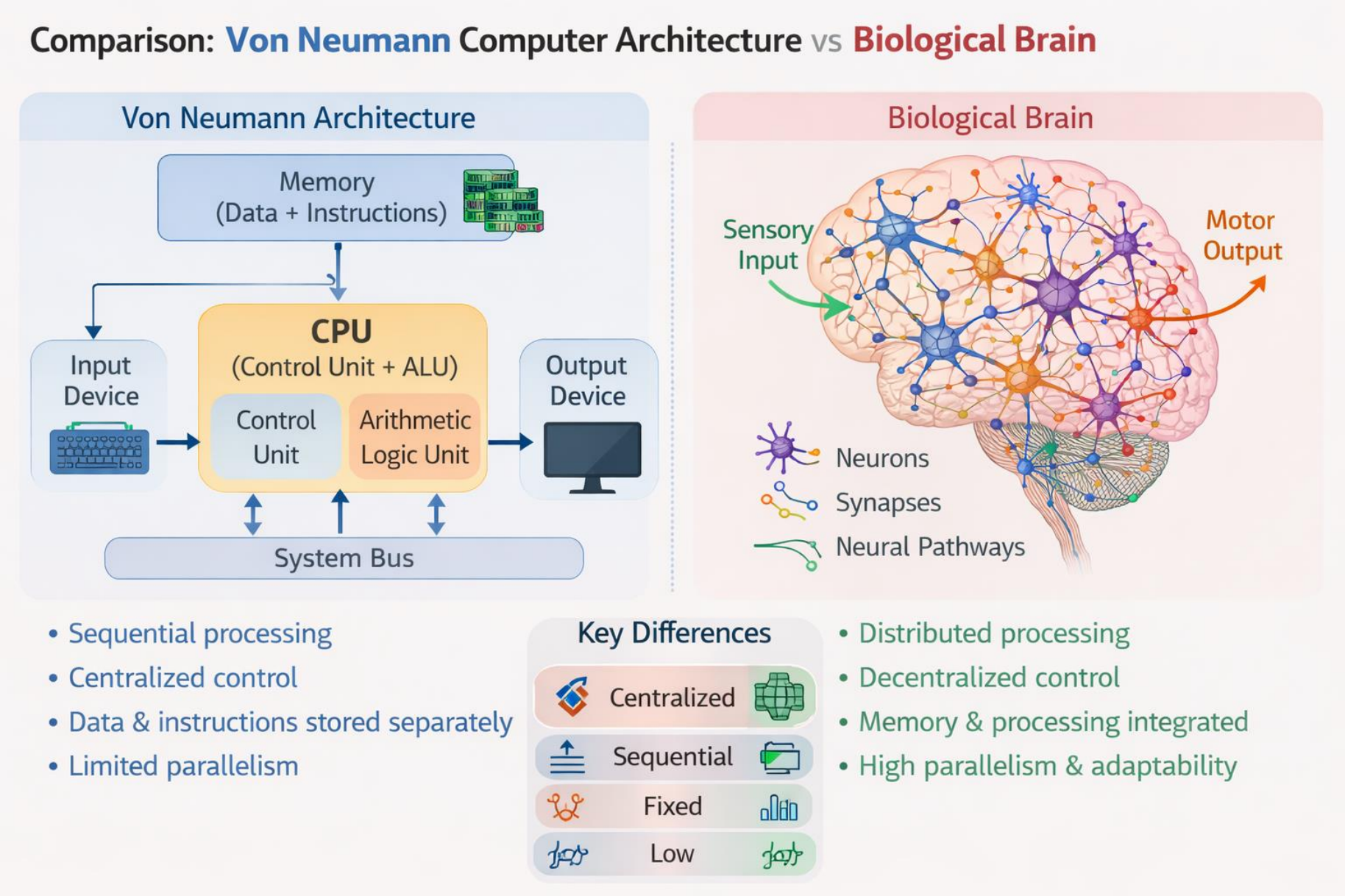
Key Features of Human Brain

- 86 Billion neurons in human brain
- On average, 1000 synapses per neuron (Purkinje cells have upto 200k synoapese)
- Electric transmission and modulation of signals
- High degree of plasticity
- Formation of new connections
- Strength of connections can modify



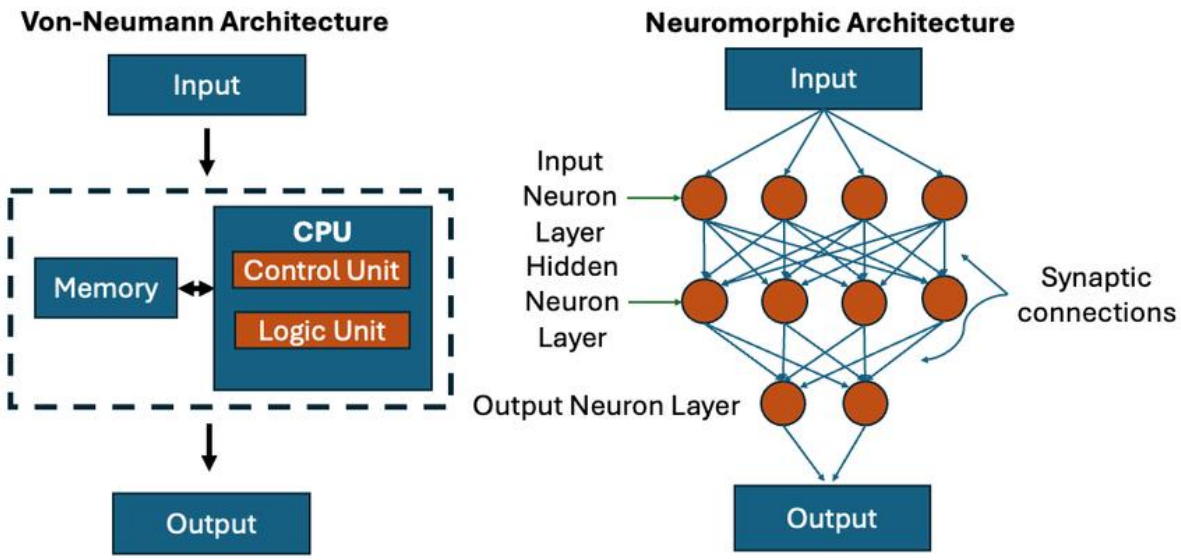
https://hessian.ai/what-ai-can-learn-from-the-human-brain/

Comparison between Human brain and a modern Computer



Comparison between a modern Computer and Neural Network

Key Attributes:	Von Neumann Architecture	Biological Neural Network
Processor	Complex, High velocity, single (or few like multi-core)	Simple, Low velocity, Huge numbers
Memory	Separated from processor, localized, not associative	Integrated, Distributed, Associative
Processing	Centralized, sequential, instructions memorized	Distributed, Parallel, self-learning
Reliability	Vulnerable	Robust



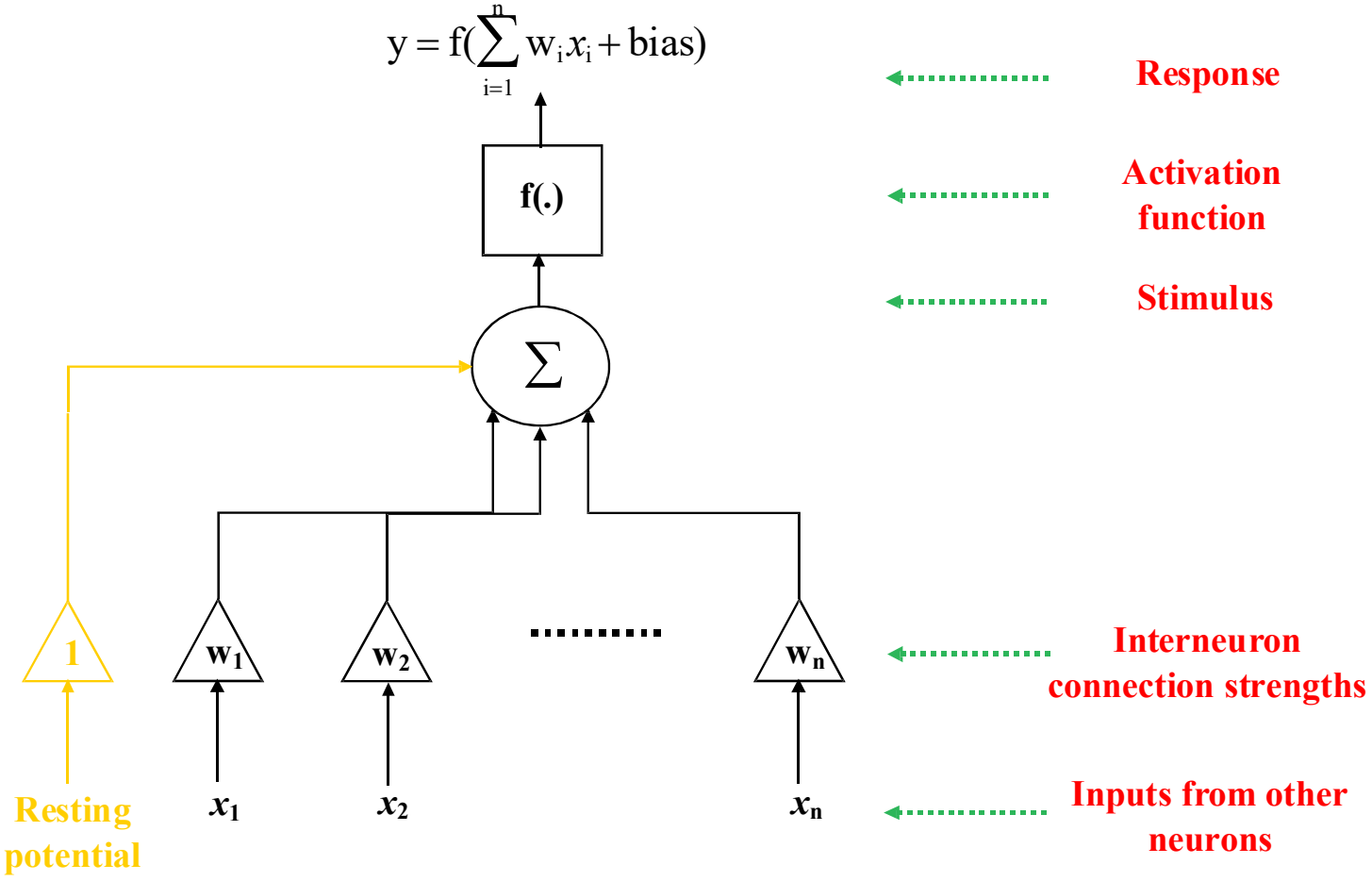
Historical developments

- Artificial neural networks (ANNs) are nonlinear systems inspired by biological neural nets.

Research in ANNs is marked by three key evolution periods:

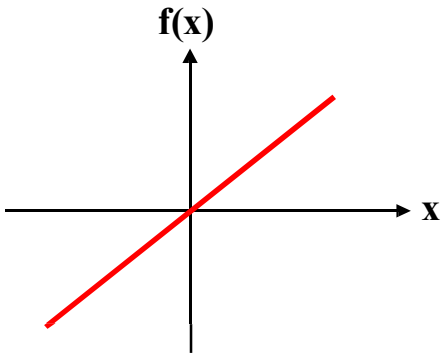
- 1940's: *McCulloch* and *Pitts* developed pioneering studies on neural nets.
- 1960's: *Rosenblatt* formulated the convergence theorem for the perceptron; *Minsky* underlined the limitations of the perceptron.
- 1980's: Interest is renewed thanks to the works of *Hopfield* and the development of the error backpropagation algorithm to train artificial multilayer perceptron neural nets

Artificial Neuron: Structure

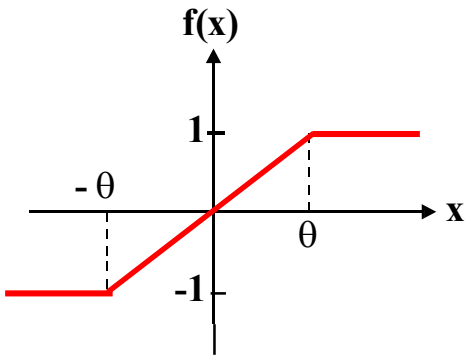


Different Activation Functions

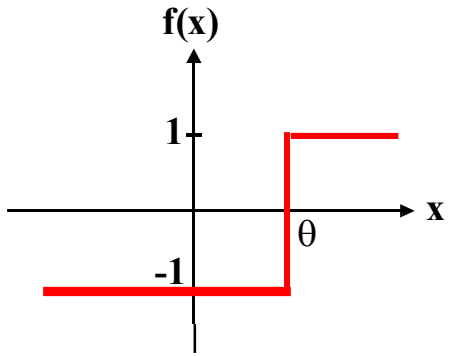
Linear



Piecewise linear

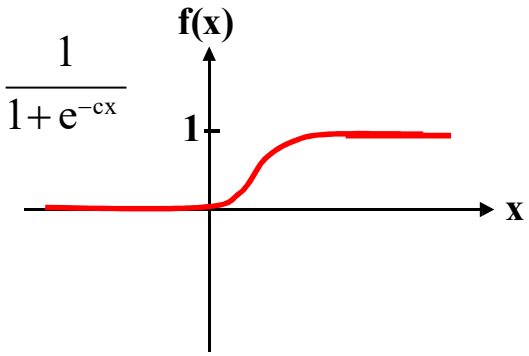


Threshold

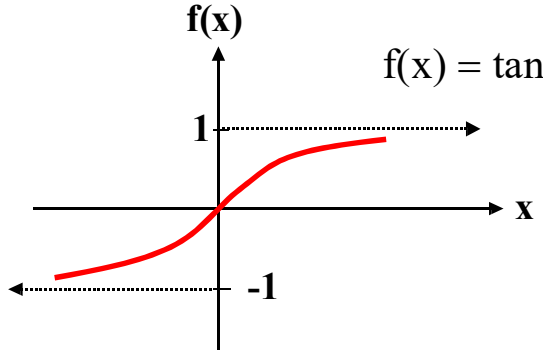


Logistic

$$f(x) = \frac{1}{1+e^{-cx}}$$



Hyperbolic tangent



$$f(x) = \tanh(cx/2) = \frac{1-e^{-cx}}{1+e^{-cx}}$$

Commonly used AF in DL

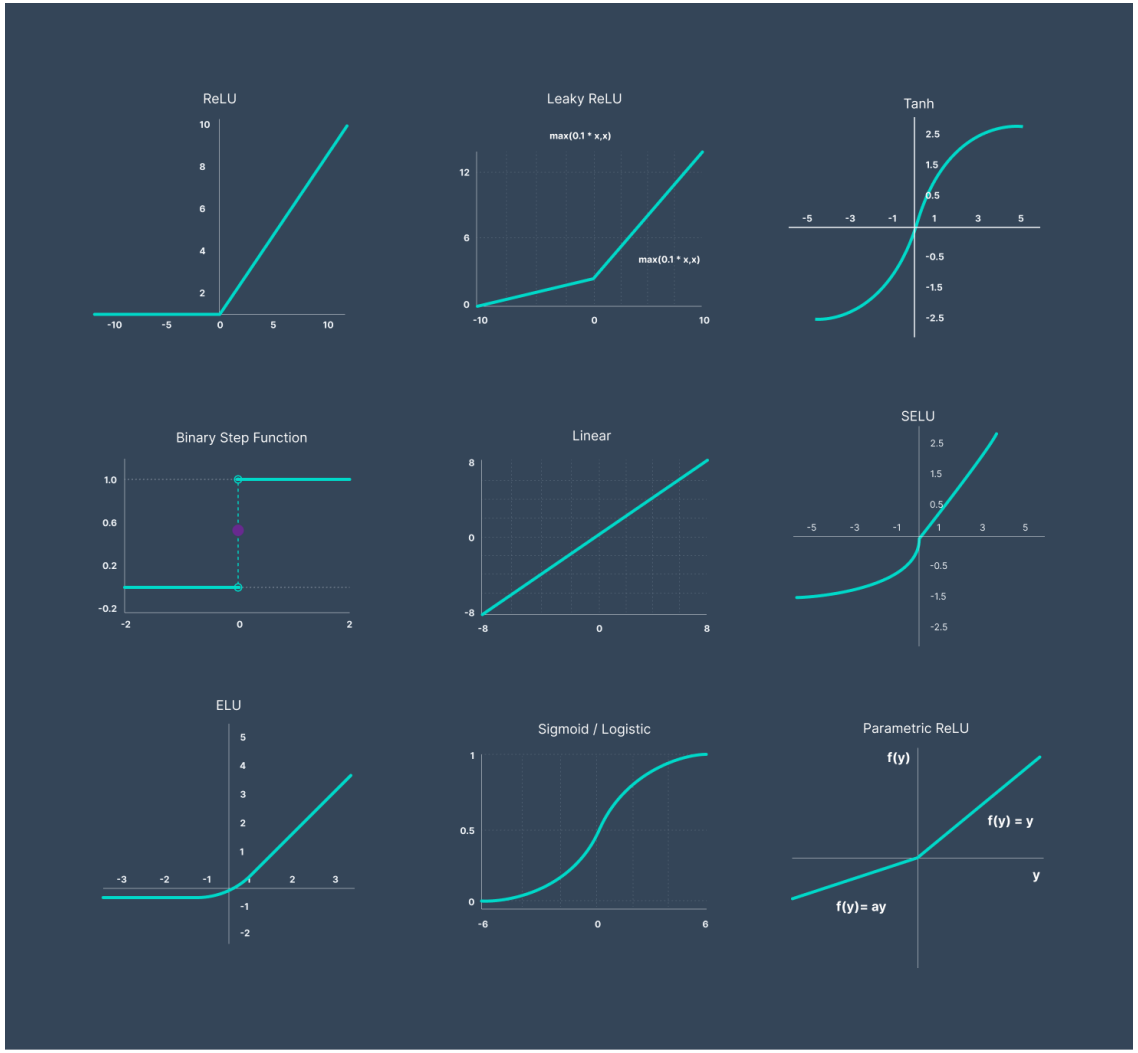
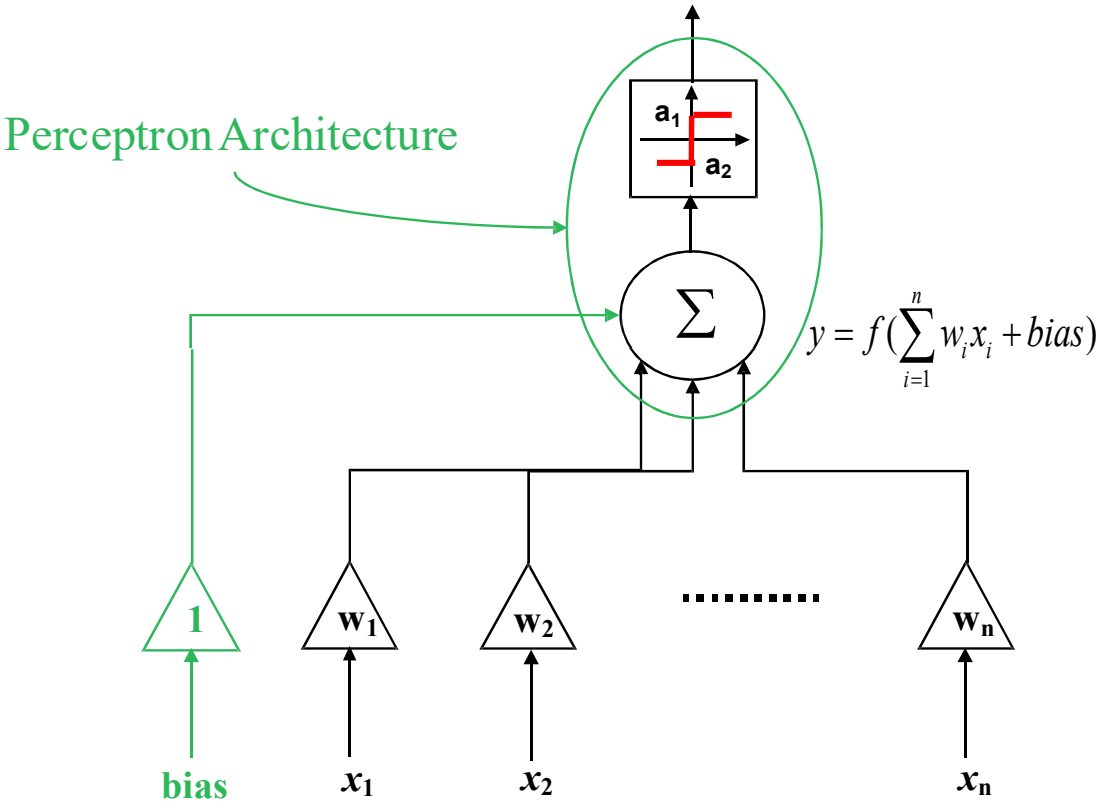


Figure courtesy: https://www.v7labs.com/blog/neural-networks-activation-functions

Perceptron



$$y = f\left(\sum_{i=1}^{n} w_i x_i + bias\right)$$

Perceptron

Given an n-dimensional input feature vector $\mathbf{x}$, the perceptron performs as a supervised binary classifier based on a linear discriminant function. Its output is:

$$y = f(\mathbf{x}) = \begin{matrix} a_1 > 0, & \mathbf{x} \in \omega_1 \\ a_2 \leq 0, & \mathbf{x} \in \omega_2 \end{matrix}$$

A critical issue for artificial neurons is how to train them, which means in the case of the perceptron how to estimate the weight vector and the bias.

Artificial Neural Networks

An ANN is composed of an interconnection of numerous artificial neurons.

Interconnecting neurons aims at increasing the capability of modeling complex input-output relationships.

The choice of the interconnection architecture and of the activation functions plays a key role in the definition of the neural net.

A single perceptron works like a linear function. It works well for linearly separably data. However, for non-linearly separable data, we need more complex structure.

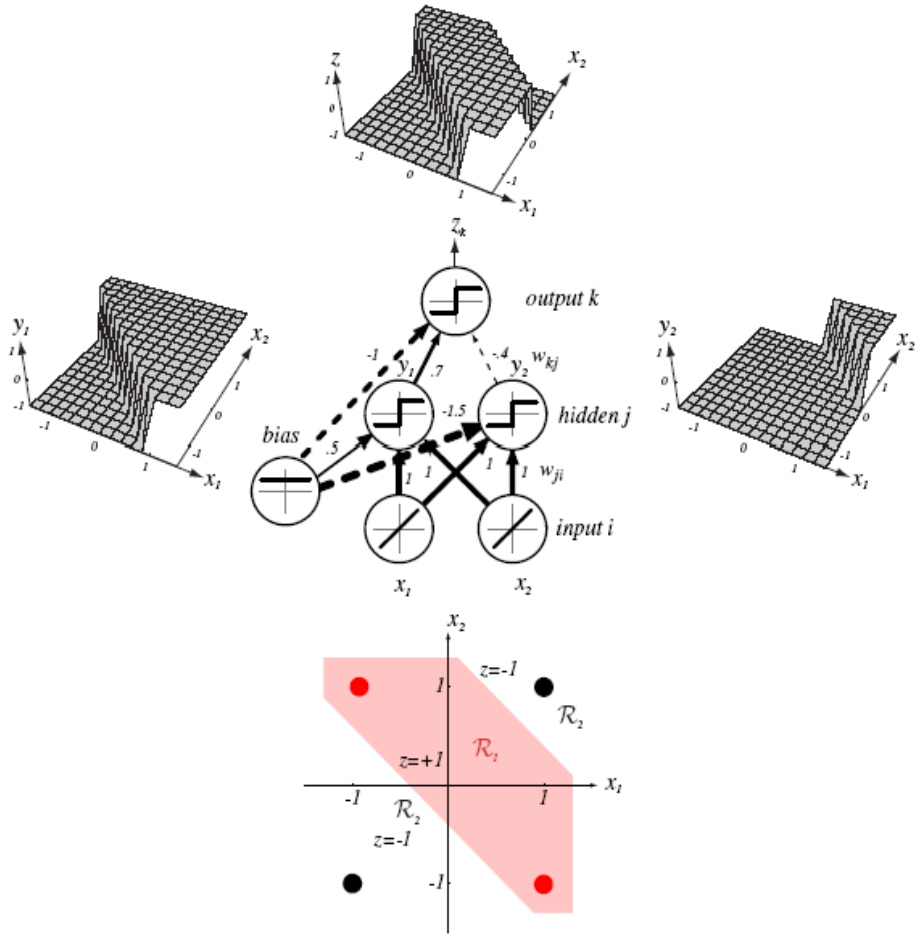
Example – XOR problem

- A typical example of non-linearly separable function is the XOR. This function takes two input arguments with values in {-1,1} and returns one output in {-1,1}, as specified in the following table:

x_1	x_2	x_1 xor x_2
-1	-1	**-1**
-1	1	**1**
1	-1	**1**
1	1	**-1**

- If we think at -1 and 1 as encoding of the truth values false and true, respectively, then XOR computes the logical exclusive or, which yields true if and only if the two inputs have different truth values.

Artificial Neural Networks



Exclusive-OR can not be solved with a single perceptron.

However, the problem can be solved by a three layer network.

Multilayer Layer Perceptron

1. The forward interconnection of several layers of perceptrons leads to an architecture, which takes the name of multilayer perceptron (MLP) neural networks.

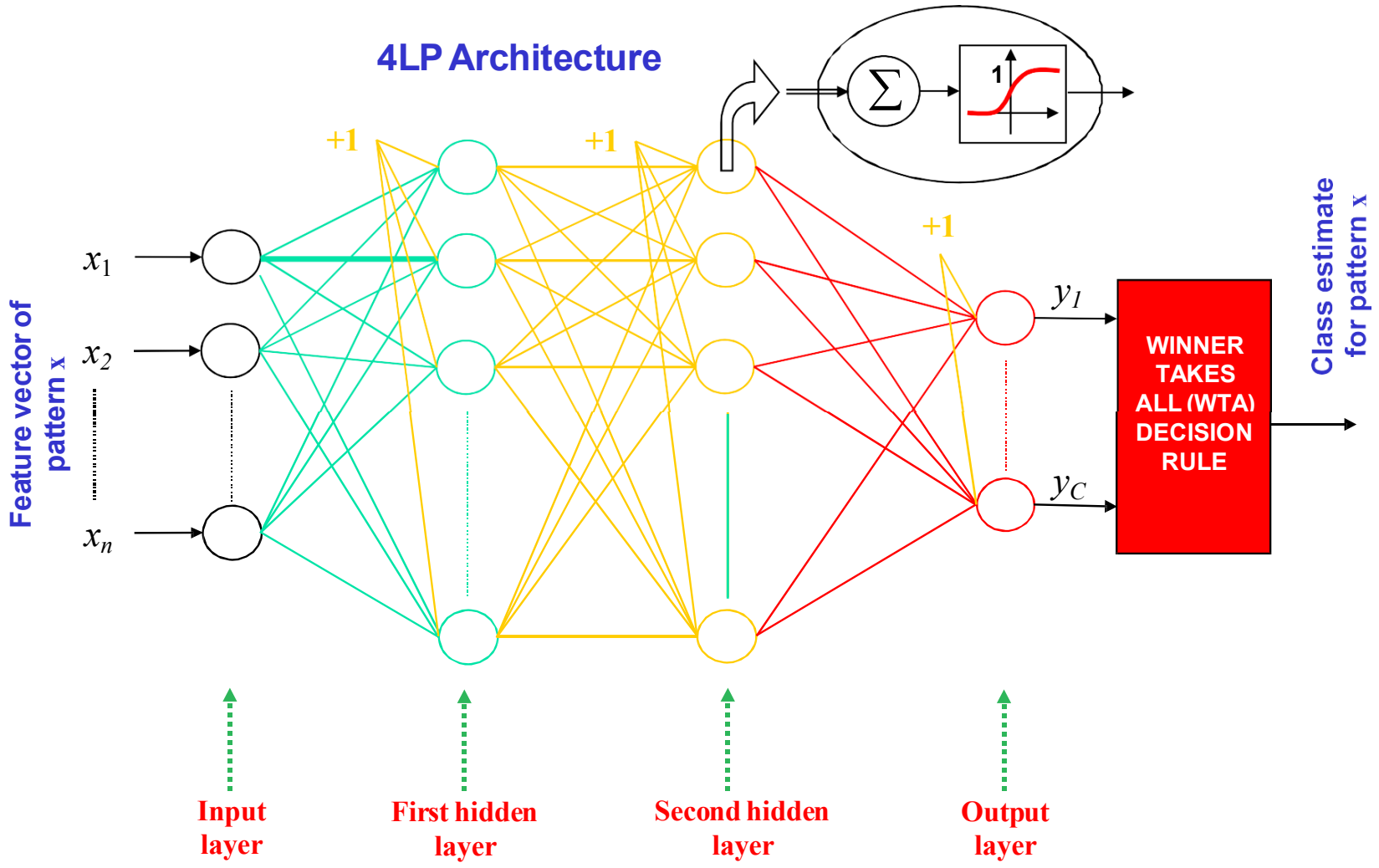
2. MLPs can be used as universal approximators.

3. In classification problems, they allow modeling nonlinear discriminant functions.

4. They started to become extremely popular in the 90's.

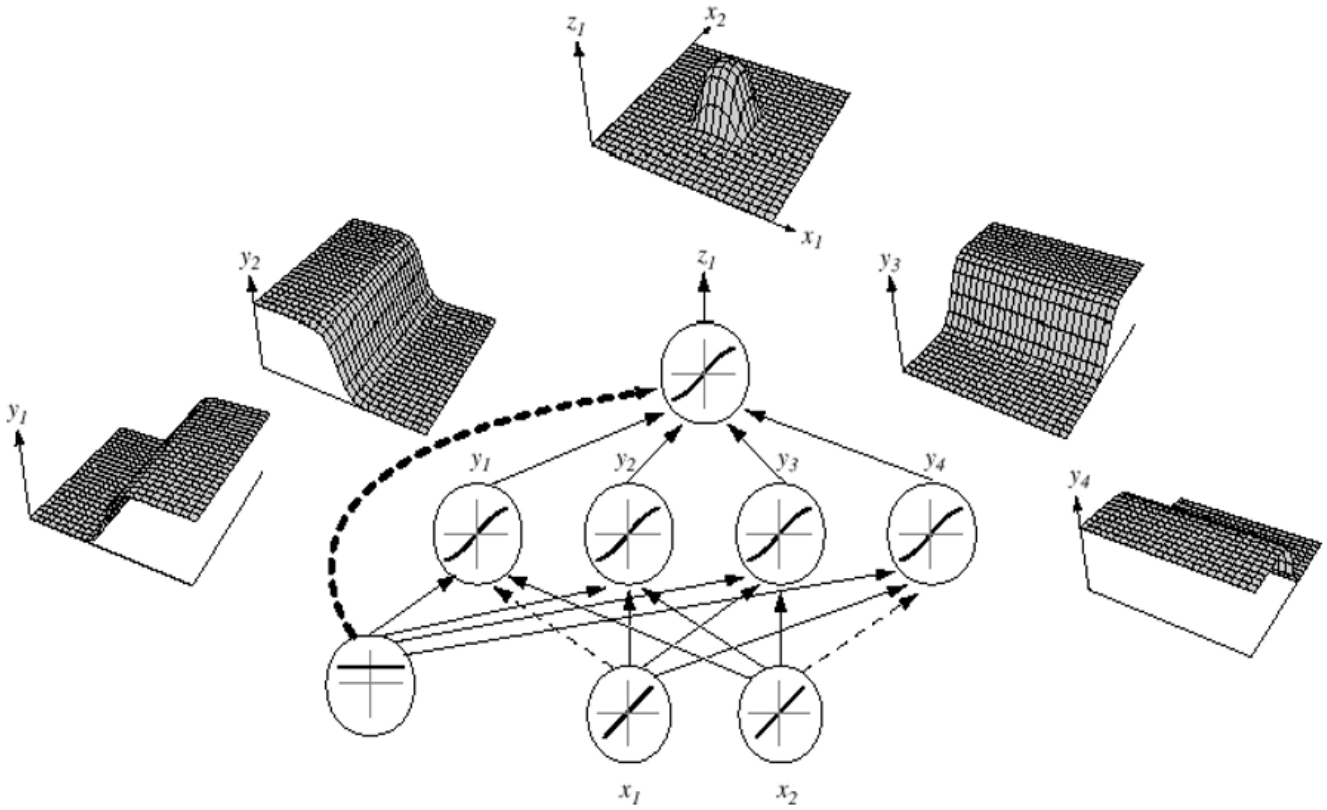
5. They have been used in numerous application fields such as identification and control, game-playing and decision making, pattern recognition, sequence recognition, medical diagnosis, financial applications and data mining.

Multilayer Layer Perceptron

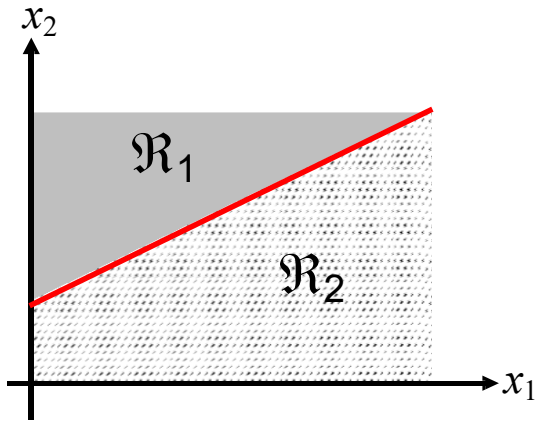


Multi Layer Perceptron

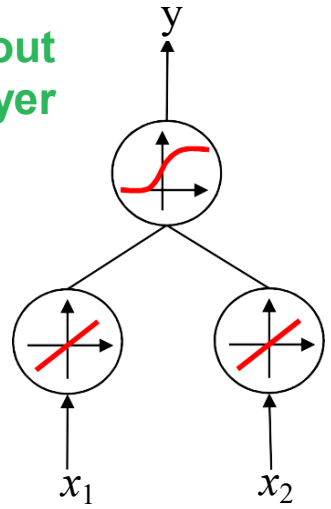
- Example of a 2-4-1 MLP along with the response functions at different units

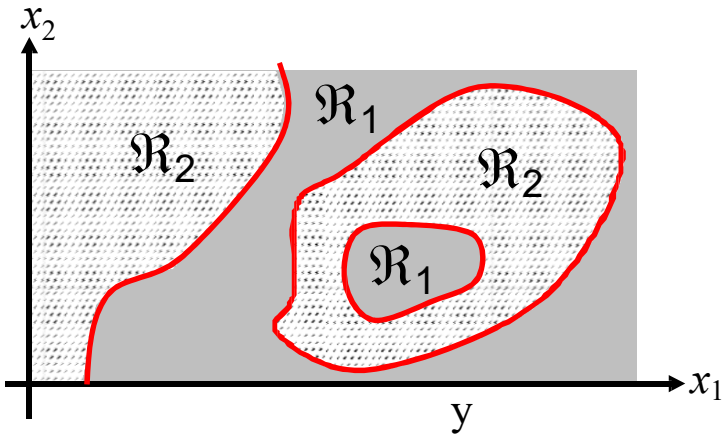


MLP: Decision Regions

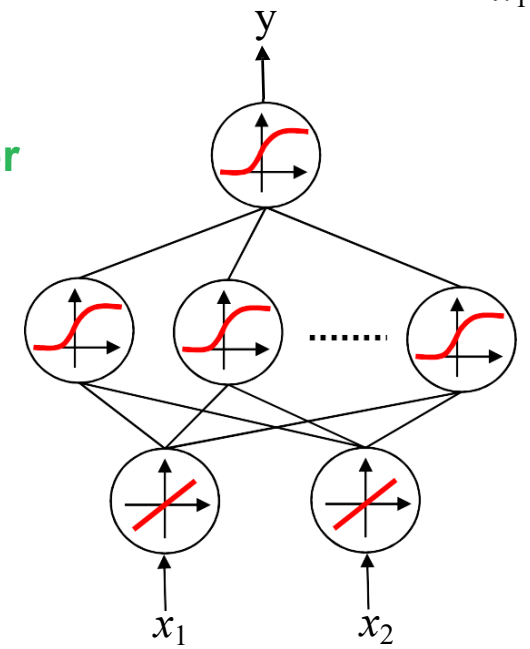


MLP without hidden layer





MLP with hidden layer



MLP: Training

- As for all supervised classifiers, one of the most important issue with MLP classifiers is how to train it.
- Training a MLP neural net means finding an opportune architecture and the related weight and bias values so that to perform the desired classification task.
- The highly nonlinear nature of MLPs makes it not trivial to find an analytical solution to the problem.

Therefore, one has to resort to numerical optimizers.

- Another problem is the number of weights and biases to optimize, as underlined by the following example:

MLP with:
- **10 input neurons**
- **20 hidden neurons**
- **5 output neurons**

→ **325 weights (+biases)!**

MLP: Backpropagation Algorithm

- Which connection weights must be modified, and by how much, to perform correctly the desired classification task.

- To put it another way, how do we know which connection is responsible for the greatest contribution to the error in the output.

- Clearly, we must use an algorithm which efficiently modifies the different connection weights to minimize the errors at the output.

- One algorithm which has hugely contributed to neural network fame is the iterative backpropagation algorithm.

MLP: Backpropagation Algorithm

It is a gradient-based search method which allows finding a (local) minimum of the sum of squared error criterion:

$$E_T = \frac{1}{2}\sum_{i=1}^{N}(\mathbf{y}_i - \mathbf{t}_i)^2$$

Total number of training samples

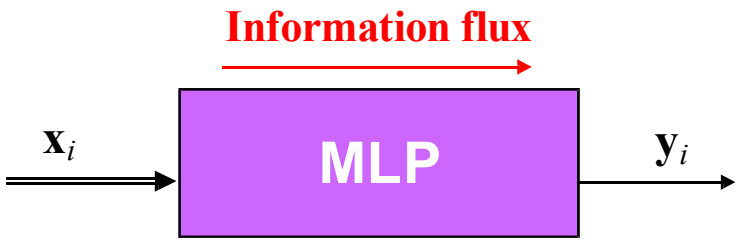
Desired output (target) value for the *i*-th sample

Output value obtained by the MLP for the *i*-th sample

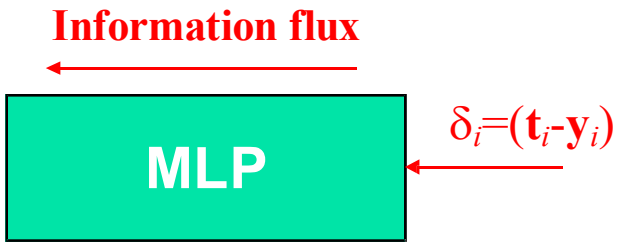
MLP: Backpropagation Algorithm

It works in three phases:

- **Forward propagation phase**



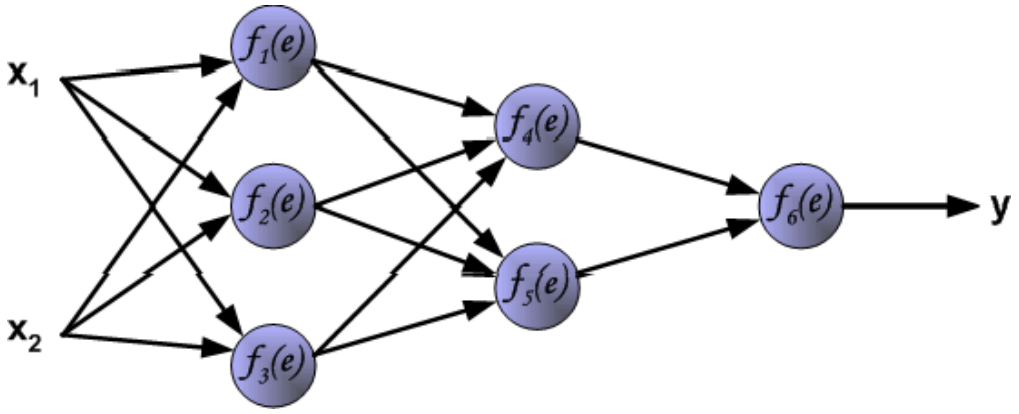
- **Backward propagation phase**



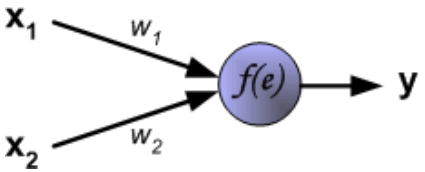
- **Weight updating phase**

Backpropagation: Illustration

Let us consider a 2-3-2-1 MLP without bias connections.



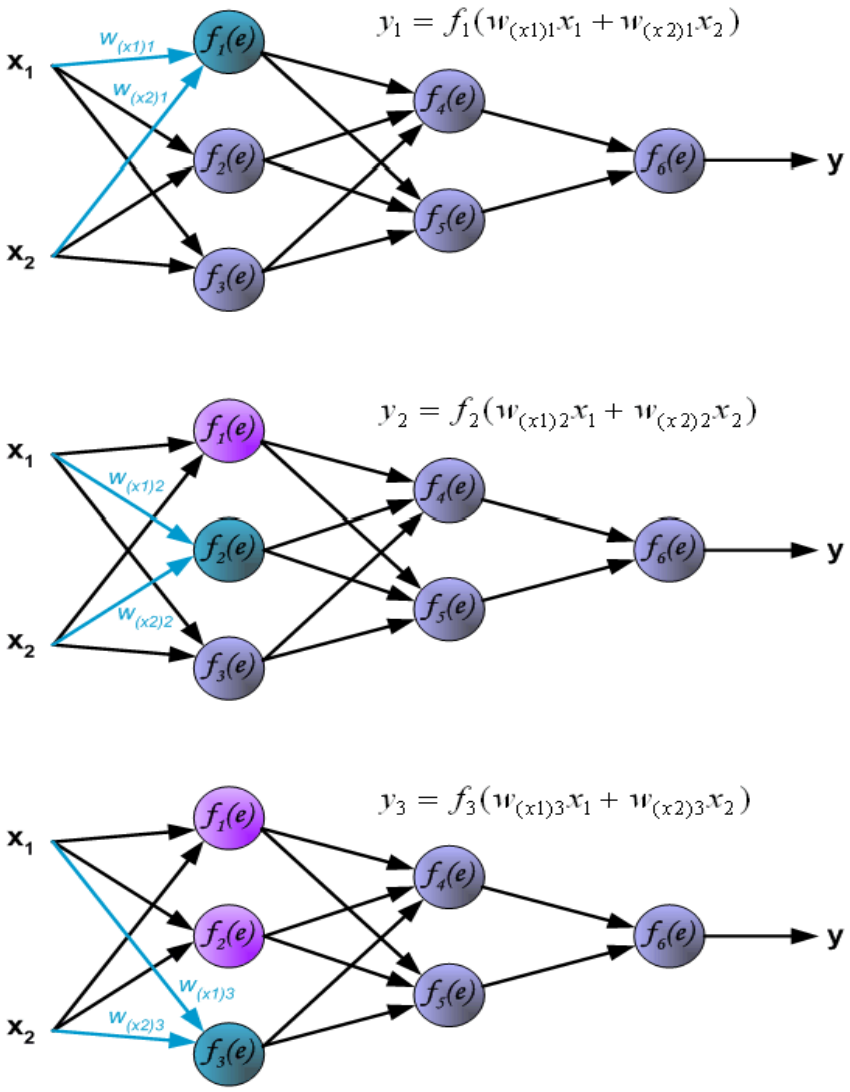
'*e*' is the adder output signal:



$e = w_1x_1 + w_2x_2$

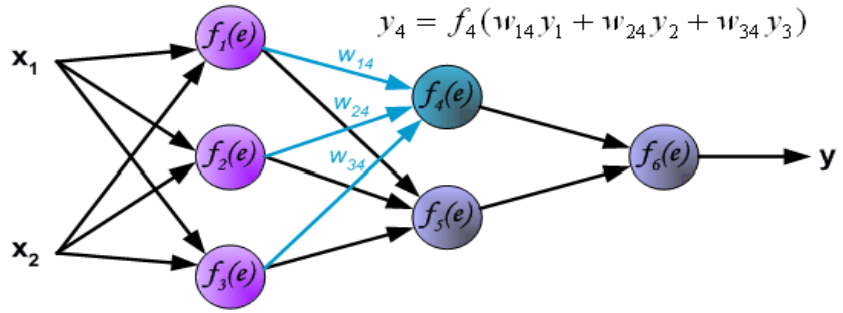
Backpropagation: Illustration

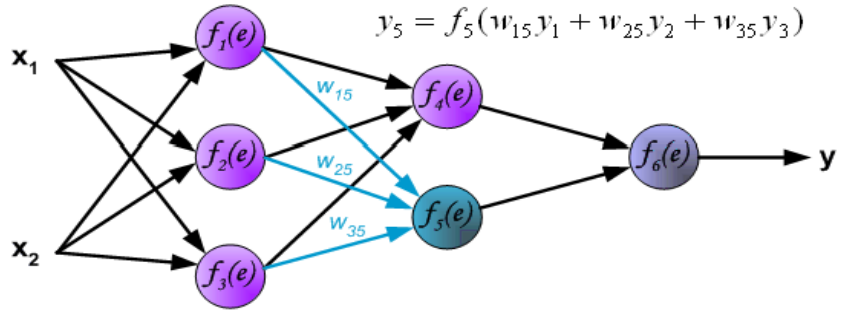
Forward propagation from input to 1st hidden layers



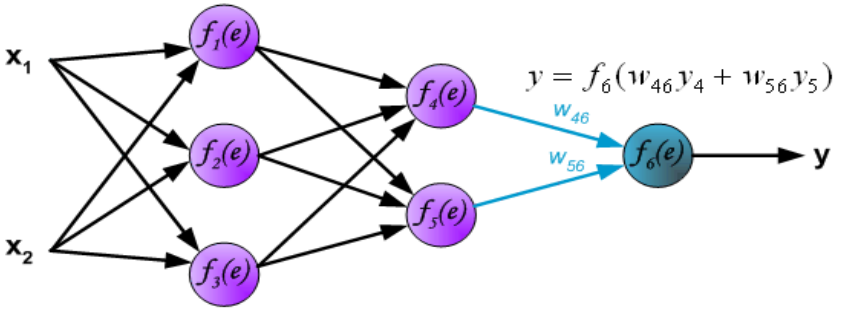
Backpropagation: Illustration

Forward propagation from 1st to 2nd hidden layers



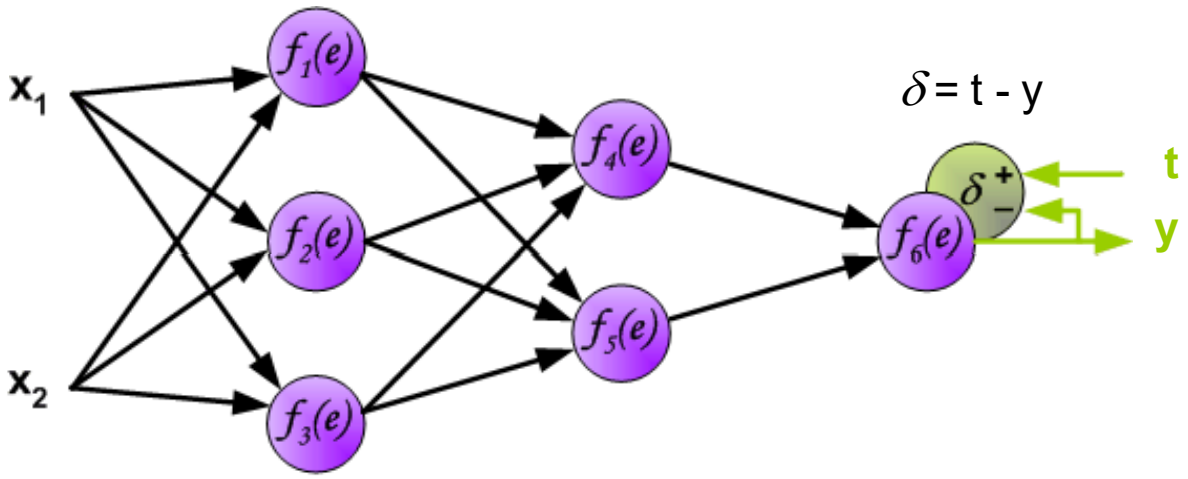


Forward propagation from 2nd hidden to output layers



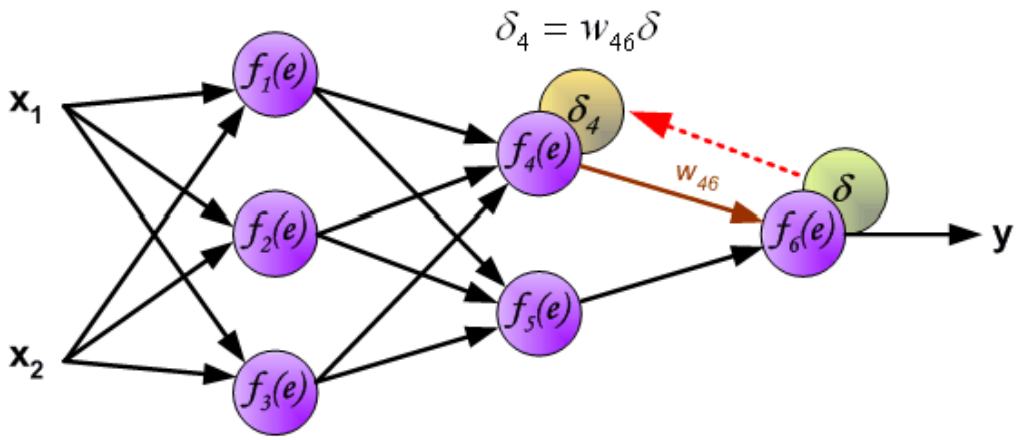
Backpropagation: Illustration

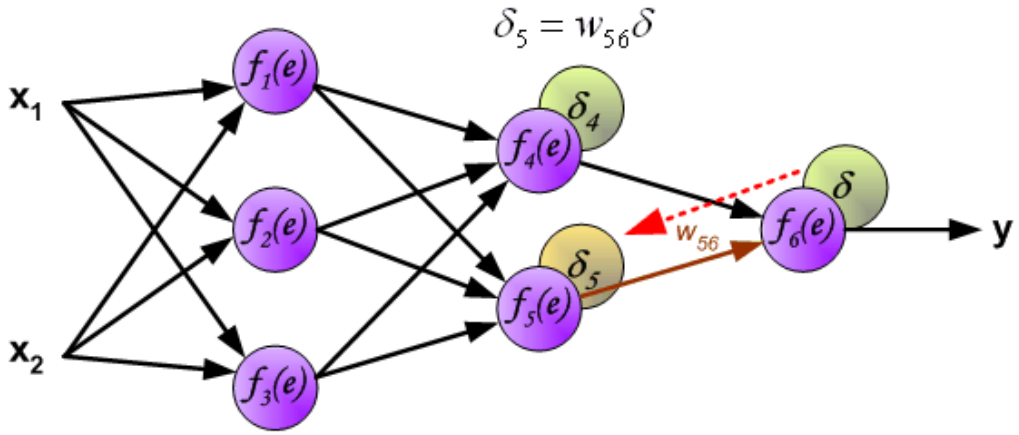
Computation of output error



Backpropagation: Illustration

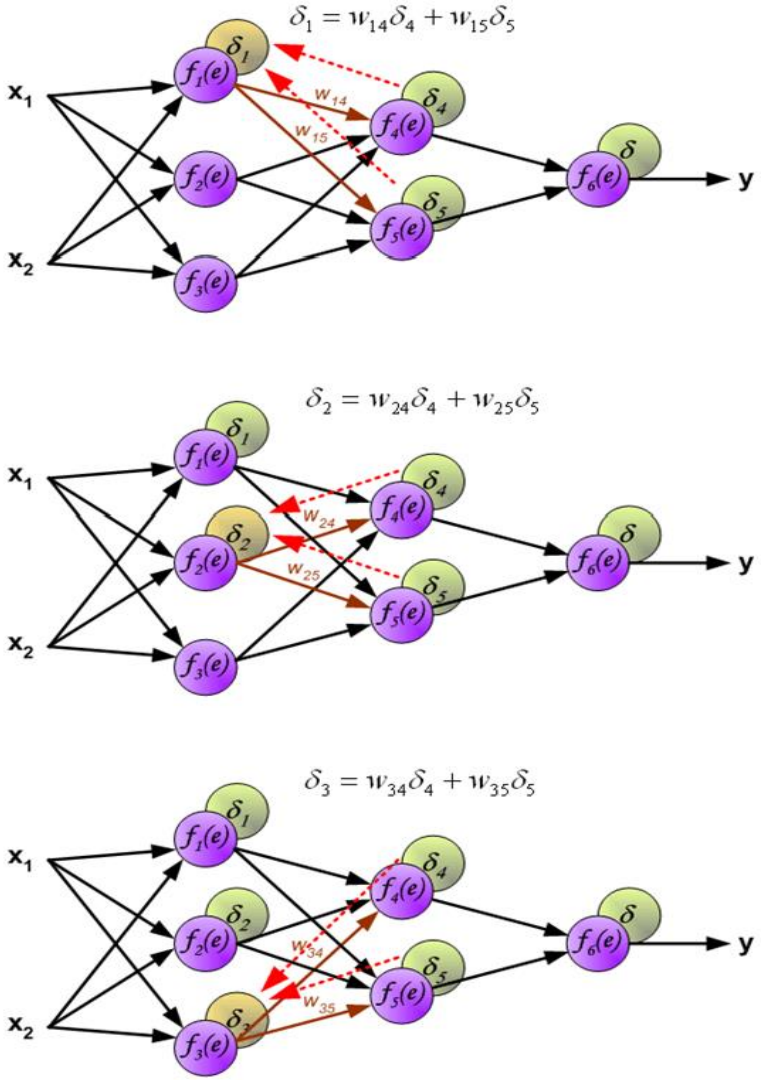
Backward propagation from output to 2nd hidden layers





Backpropagation: Illustration

Backward propagation from 2nd to 1st hidden layers

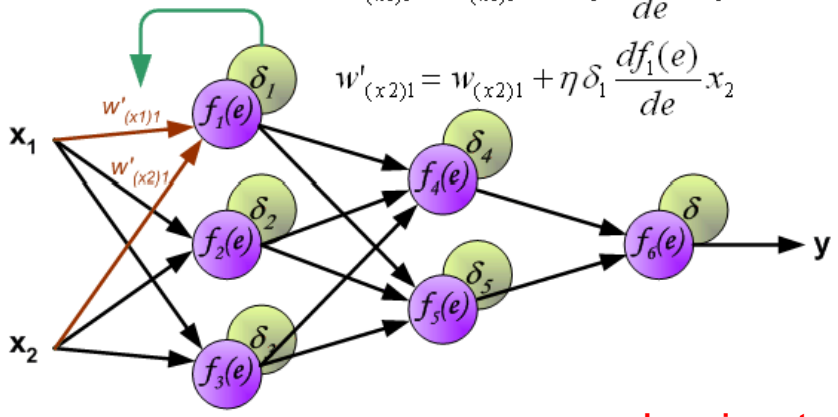


Backpropagation: Illustration

Weight updating

$$w'_{(x1)1} = w_{(x1)1} + \eta\delta_1 \frac{df_1(e)}{de} x_1$$

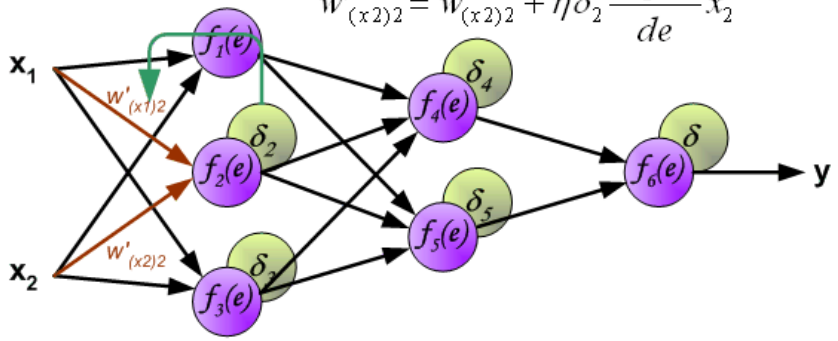
$$w'_{(x2)1} = w_{(x2)1} + \eta\delta_1 \frac{df_1(e)}{de} x_2$$



Learning rate

$$w'_{(x1)2} = w_{(x1)2} + \eta\delta_2 \frac{df_2(e)}{de} x_1$$

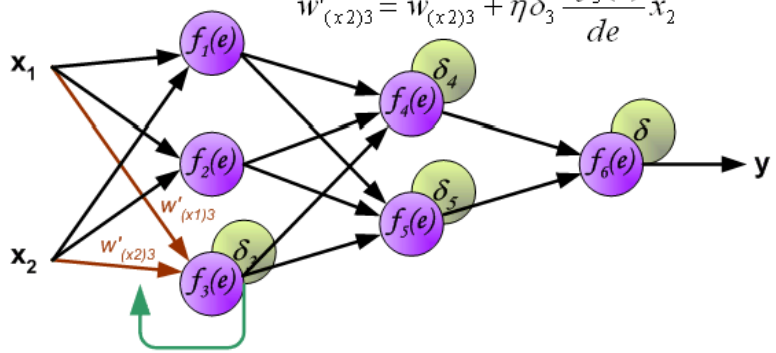
$$w'_{(x2)2} = w_{(x2)2} + \eta\delta_2 \frac{df_2(e)}{de} x_2$$



Backpropagation: Illustration

$$w'_{(x1)3} = w_{(x1)3} + \eta\delta_3 \frac{df_3(e)}{de} x_1$$

$$w'_{(x2)3} = w_{(x2)3} + \eta\delta_3 \frac{df_3(e)}{de} x_2$$

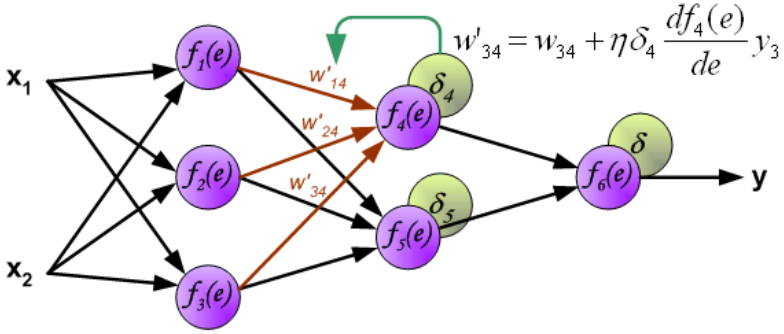


Weight updating

$$w'_{14} = w_{14} + \eta\delta_4 \frac{df_4(e)}{de} y_1$$

$$w'_{24} = w_{24} + \eta\delta_4 \frac{df_4(e)}{de} y_2$$

$$w'_{34} = w_{34} + \eta\delta_4 \frac{df_4(e)}{de} y_3$$



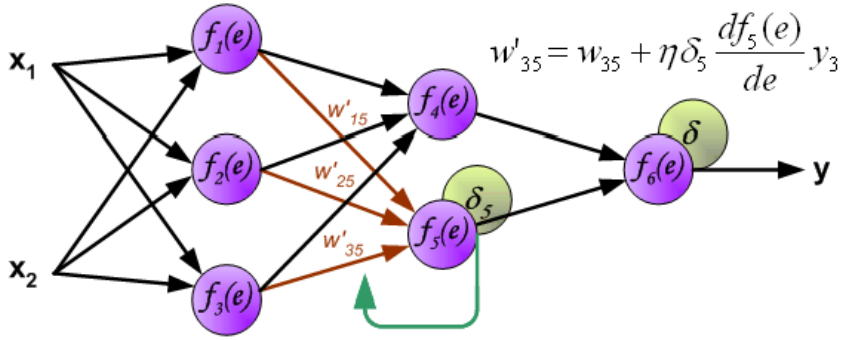
Backpropagation: Illustration

$$w'_{15} = w_{15} + \eta \delta_5 \frac{df_5(e)}{de} y_1$$

$$w'_{25} = w_{25} + \eta \delta_5 \frac{df_5(e)}{de} y_2$$

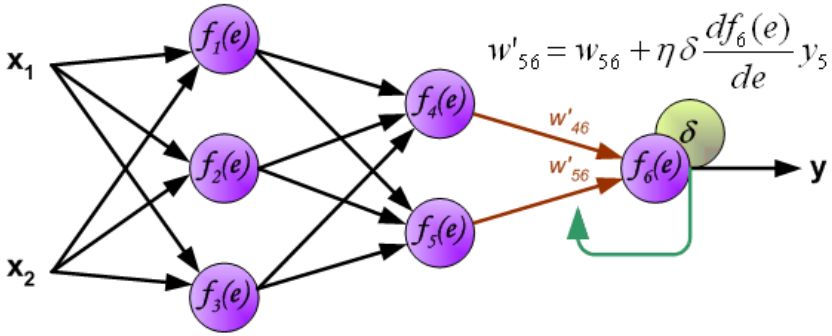
$$w'_{35} = w_{35} + \eta \delta_5 \frac{df_5(e)}{de} y_3$$

Weight updating



$$w'_{46} = w_{46} + \eta \delta \frac{df_6(e)}{de} y_4$$

$$w'_{56} = w_{56} + \eta \delta \frac{df_6(e)}{de} y_5$$



MLP for Regression: Problem Setup

- Goal: predict continuous value $y \in \mathbb{R}$
- Examples: traffic volume, anomaly score, latency
- No discrete classes

Regression: Architecture

- Define Input and hidden layers of the MLP
- Choose appropriate activation for the different perceptron in the network.
- Output layer: 1 neuron
- No activation (linear output)

Regression – Model Output

- Output: $\hat{y}$ = real number (Continous output)
- Represents estimated target value
- No probability interpretation

Regression – Loss Functions

- Mean Squared Error (MSE): (y − ŷ)^2
- Mean Absolute Error (MAE)
- Huber loss for robustness to outliers

Mean Square Error (MSE)

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Mean Absolute Error (MAE)

$$\mathrm{MAE} = \frac{\sum_{i=1}^{n}|y_i - x_i|}{n}$$

Huber Loss

$$L_\delta(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{for} |y - f(x)| \le \delta, \\ \delta\left(|y - f(x)| - \frac{1}{2}\delta\right), & \text{otherwise.} \end{cases}$$

https://en.rattibha.com/thread/1504170139792588808

Regression – Backpropagation

- Gradient depends on derivative of chosen loss
- For MSE: $\partial L/\partial z = \hat{y} - y$
- Same backpropagation mechanism as classification

MLP for Binary Classification: Problem Setup

- Goal: classify each sample into one of two classes (e.g., Normal vs Attack)
- Labels must be in {0, 1}
- Input: feature vector $x \in R^d$
- Output: probability of class 1 (attack)

Binary Classification – MLP Architecture

- Input layer: numerical / encoded categorical features
- Hidden layers: fully connected + nonlinear activation (ReLU, sigmoid, tanh)
- Output layer: single neuron
- Activation at output: Sigmoid

Binary Classification – Model Output

- Network produces a logit z (real number)
- Sigmoid converts it to probability: $\hat{y} = 1 / (1 + e^{(-z)})$
- $\hat{y} \in (0, 1)$ represents $P(\text{class} = 1 \mid x)$
- Decision rule: predict 1 if $\hat{y} \geq$ threshold (e.g., 0.5)

Binary Classification – Loss Function

- Binary Cross-Entropy (Log Loss)
- $L(y, \hat{y}) = -[\, y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y}) \,]$
- Defined only for $y \in \{0,1\}$
- Derived from Bernoulli likelihood

Binary Classification – Why Labels Must Be 0/1

- Loss formula assumes Bernoulli random variable
- Using labels such as {2,3} breaks the probabilistic interpretation
- Can lead to negative loss and incorrect gradients
- Therefore: always remap labels to {0,1}

Binary Classification: Backpropagation at Output Layer

- Key result: $\partial L/\partial z = \hat{y} - y$

- Error signal = prediction − true label

Binary Classification: Optimization Process

- Forward pass: compute predictions $\hat{y}$
- Compute loss using binary cross-entropy
- Backward pass: propagate gradients using chain rule
- Update weights using gradient descent / Adam optimizer

Binary Classification: Cybersecurity Notes

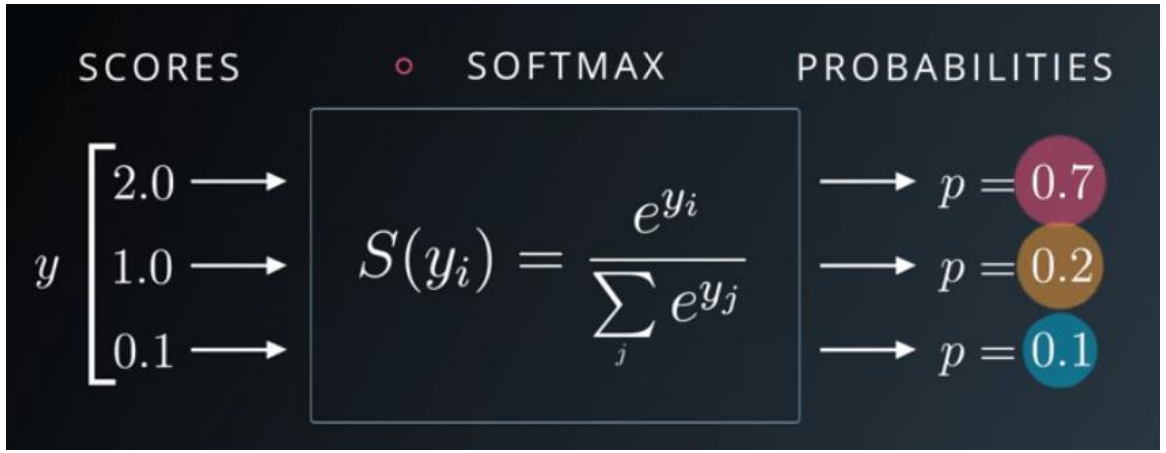
- Class imbalance is common (few attacks)
- Use weighted loss or focal loss
- Tune decision threshold for recall vs precision
- Always scale numerical features

MLP for Multi-class Classification – Problem Setup

- Goal: classify samples into $K > 2$ classes
- Example: {Normal, DoS, Probe, R2L, U2R}
- Labels: class indices $\{0, 1, \ldots, K-1\}$

Multi-class Classification – Architecture

- Same hidden layers as binary case
- Output layer: K neurons
- Activation: Softmax



https://sefiks.com/2017/11/08/softmax-as-a-neural-networks-activation-function/

Multi-class Classification: Softmax Output

- Softmax converts logits to probabilities
- $p_k = \exp(z_k) / \Sigma_j \exp(z_j)$
- Output vector sums to 1
- Prediction = $\text{argmax}(p_k)$

Multi-class Classification: Loss Function

- Categorical Cross-Entropy
- $L = -\sum_k y_k \cdot \log(p_k)$
- y_k is one-hot encoded target
- Equivalent to negative log-likelihood

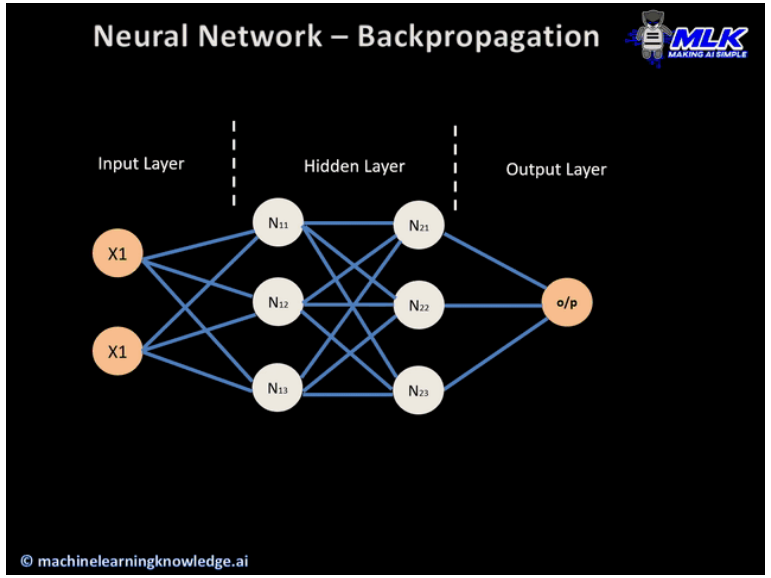
Multi-class Classification – Backpropagation

- Gradient at output layer: $p_k - y_k$
- Error distributed to hidden layers via chain rule
- Same training pipeline as binary classification

Binary vs Multi-class Classification

- Binary: 1 output neuron + sigmoid + BCE loss
- Multi-class: K output neurons + softmax + categorical CE loss
- Both use gradient descent and backpropagation
- Label encoding differs

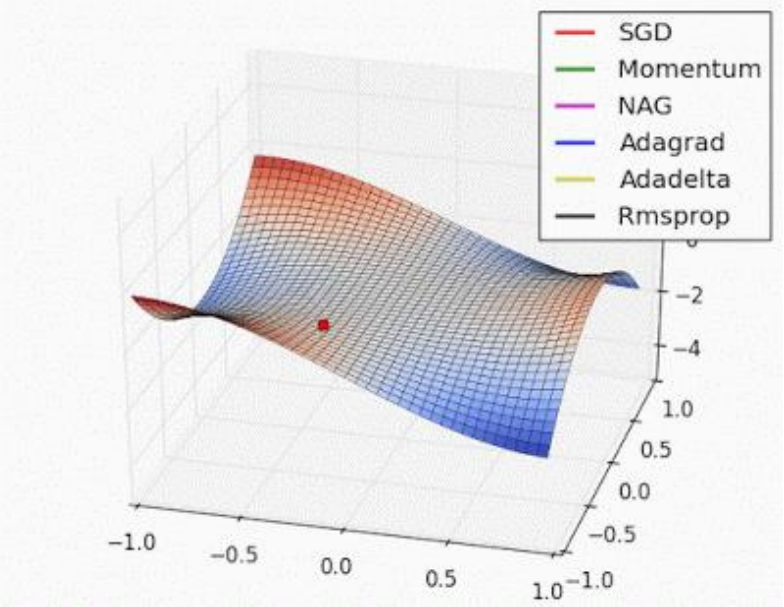
MLP: Backpropagation Algorithm



The backpropagation algorithm is iterated up to convergence.

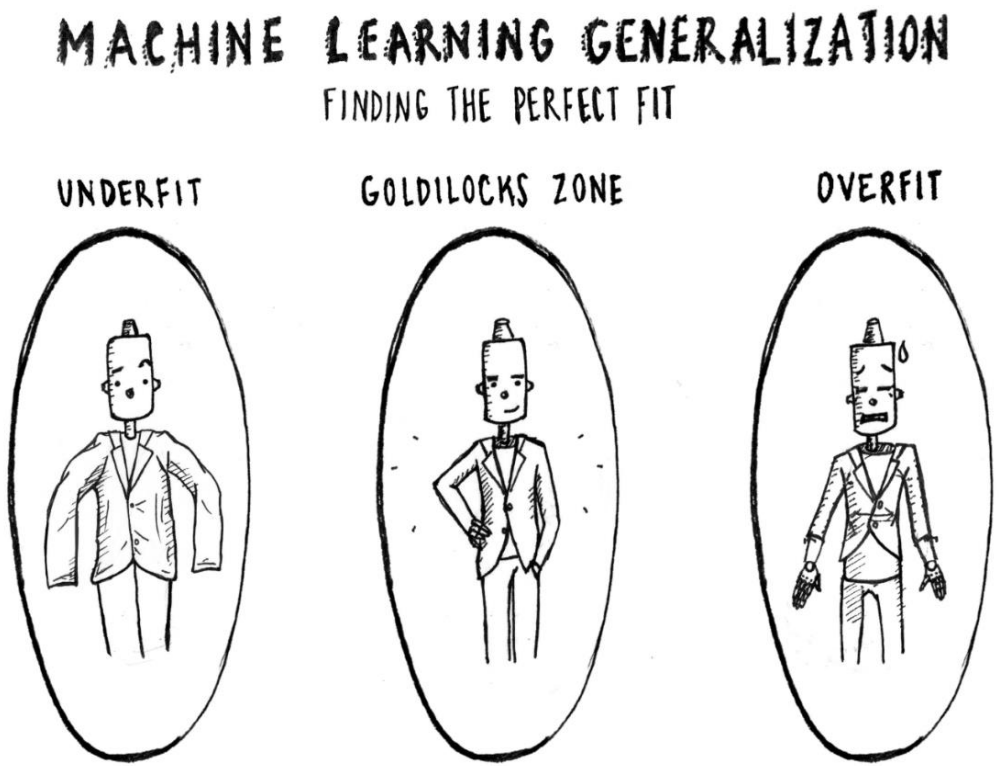
The convergence velocity depends on the:

- Complexity of the considered classification problem;
- Complexity of the approximating function;
- Value of the learning rate.



Overfitting & Underfitting

- The cause of poor performance in machine learning is either due to overfitting or underfitting.

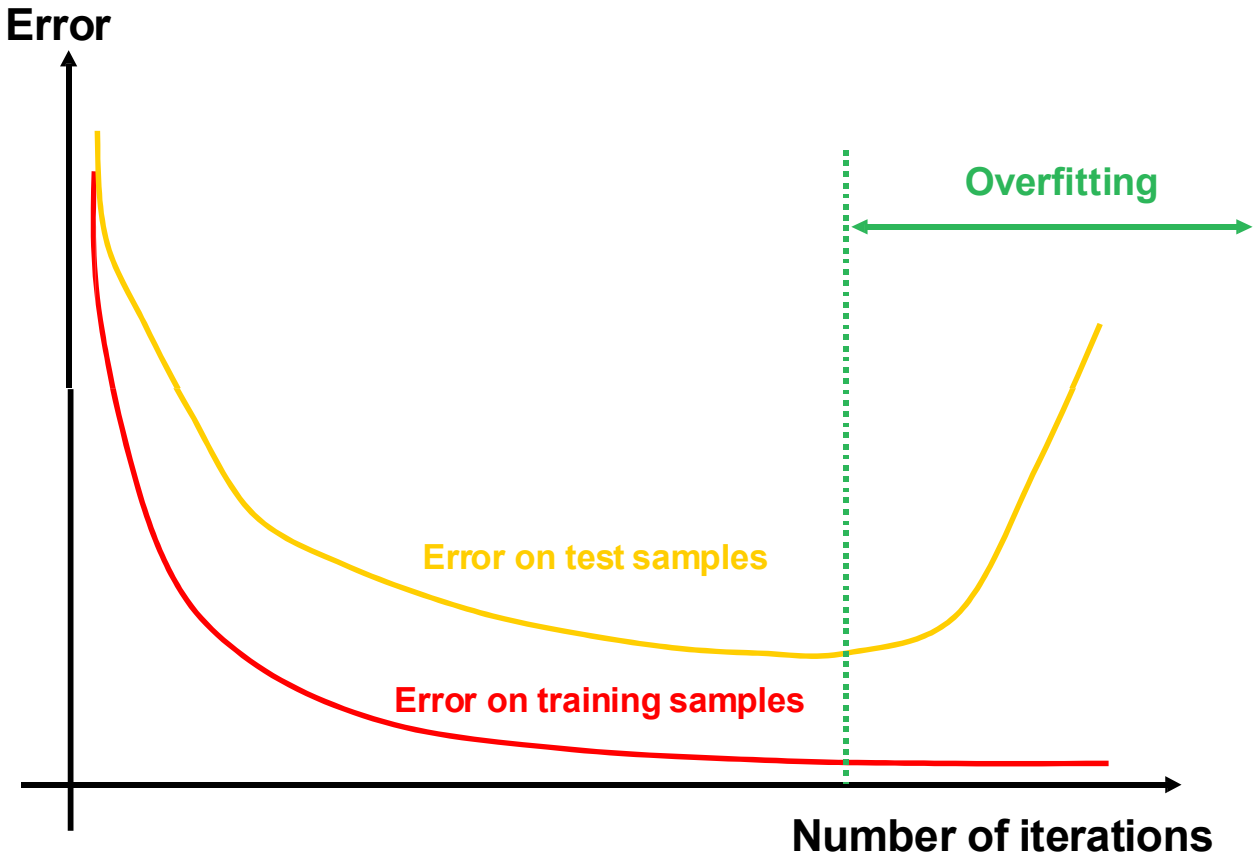


https://www.analyticsvidhya.com/blog/2020/02/underfitting-overfitting-best-fitting-machine-learning/

MLP: Overfitting

- Overfitting occurs when the machine learning algorithm models the training data too well.

- In overfitting, the model essentially learns the detail and noise in the training data to the extent that it negatively impacts the performance of the model on new data.



MLP: Underfitting

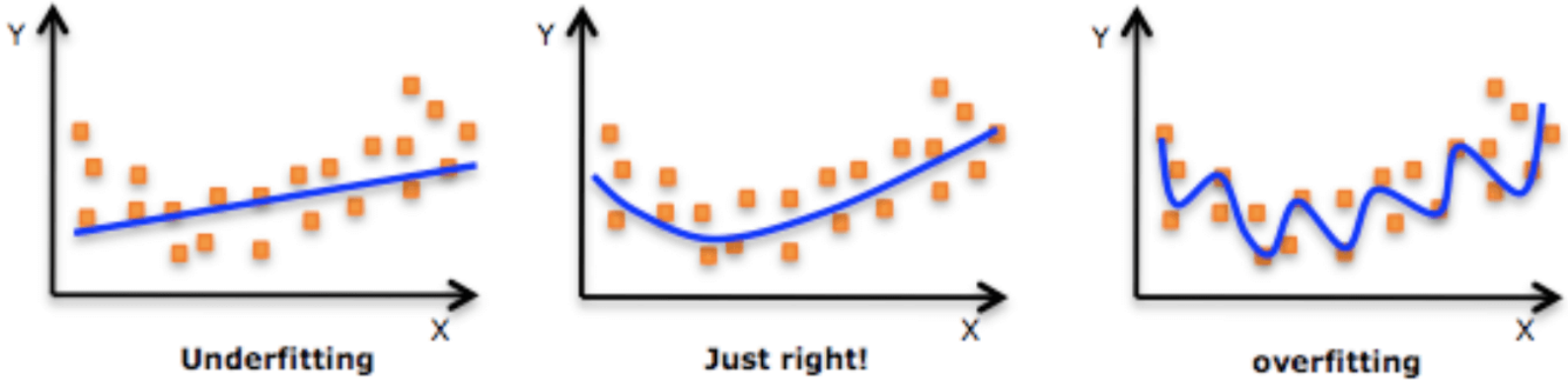
- Underfitting occurs when the machine learning algorithm is not complex enough to accurately capture relationships between a dataset's features and a target variable.

- An underfitted model results in problematic or erroneous outcomes on new data, or data that it wasn't trained on, and often performs poorly even on training data.

Underfitting: The model is too simple and doesn't learn enough patterns. It has high bias and low variance.

Overfitting: The model learns too much from the training data, including noise. It has low bias and high variance.

Just Model: It finds the right balance between bias and variance, generalizing well to new data.



https://www.datarobot.com/wiki/underfitting/

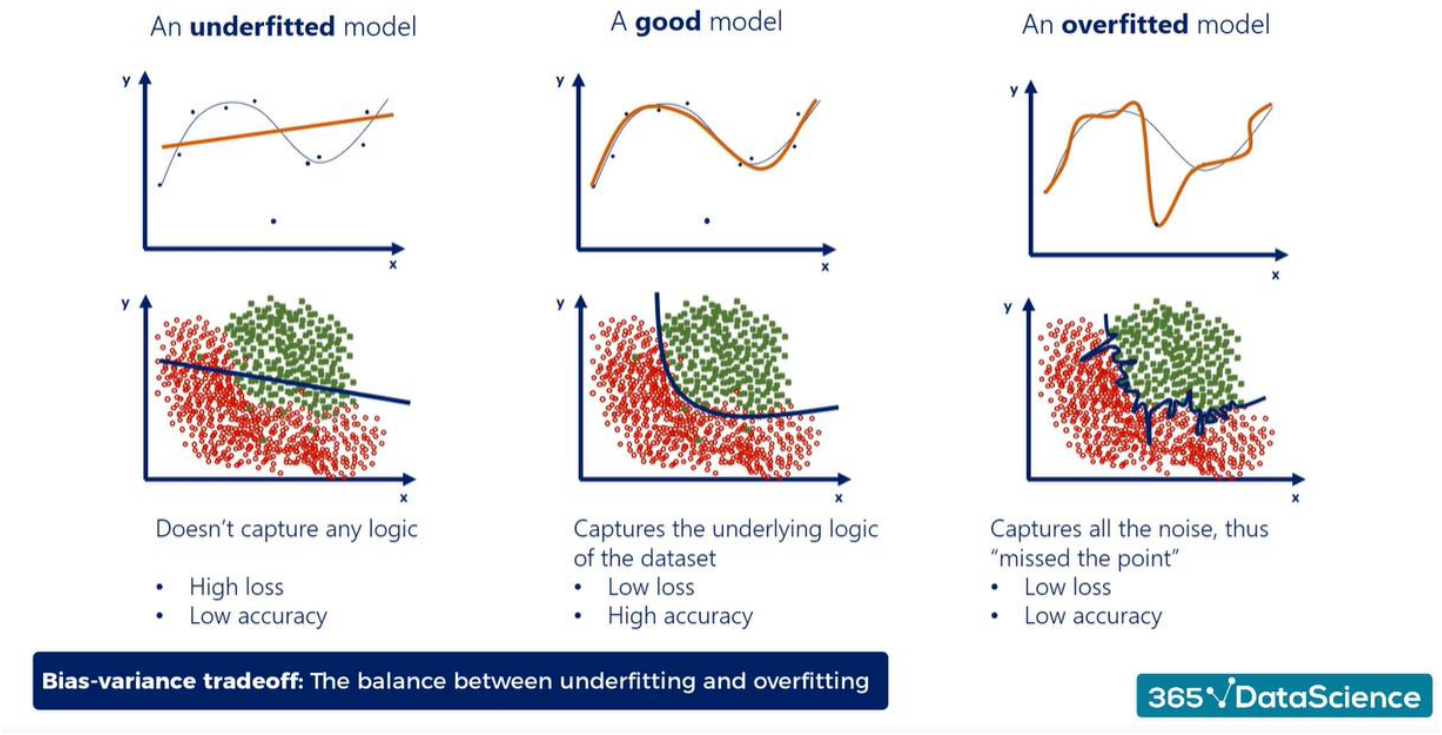
Bias-Variance Tradoff

In machine learning, bias and variance are two sources of error that affect a model's performance:

- **Bias:** Error due to overly simplistic assumptions in the learning algorithm. A high-bias model may underfit the data, meaning it doesn't capture the underlying patterns well.

- **Variance:** Error due to excessive sensitivity to the training data. A high-variance model may overfit, meaning it learns noise and performs poorly on unseen data.

A good machine learning model balances bias and variance to achieve optimal generalization.



Bias-Variance

Bias (Too Simple):

If the student always uses the same basic method, even for tricky problems, they will make mistakes. This is like a model that is too simple and misses important patterns—it has high bias (underfitting).

- Example: A student who always gives the same basic answer without fully understanding the question. This leads to underfitting.

Variance (Too Complex):

If the student memorizes every math problem but struggles with new ones, they have learned too much noise. This is like a model that only works well on training data but fails on new data—it has high variance (overfitting).

- Example: A student who memorizes every question but struggles when faced with new ones. This leads to overfitting.

A good model is like a smart student who understands the concepts, not just memorizes answers. It balances bias and variance to perform well on both seen and unseen data.

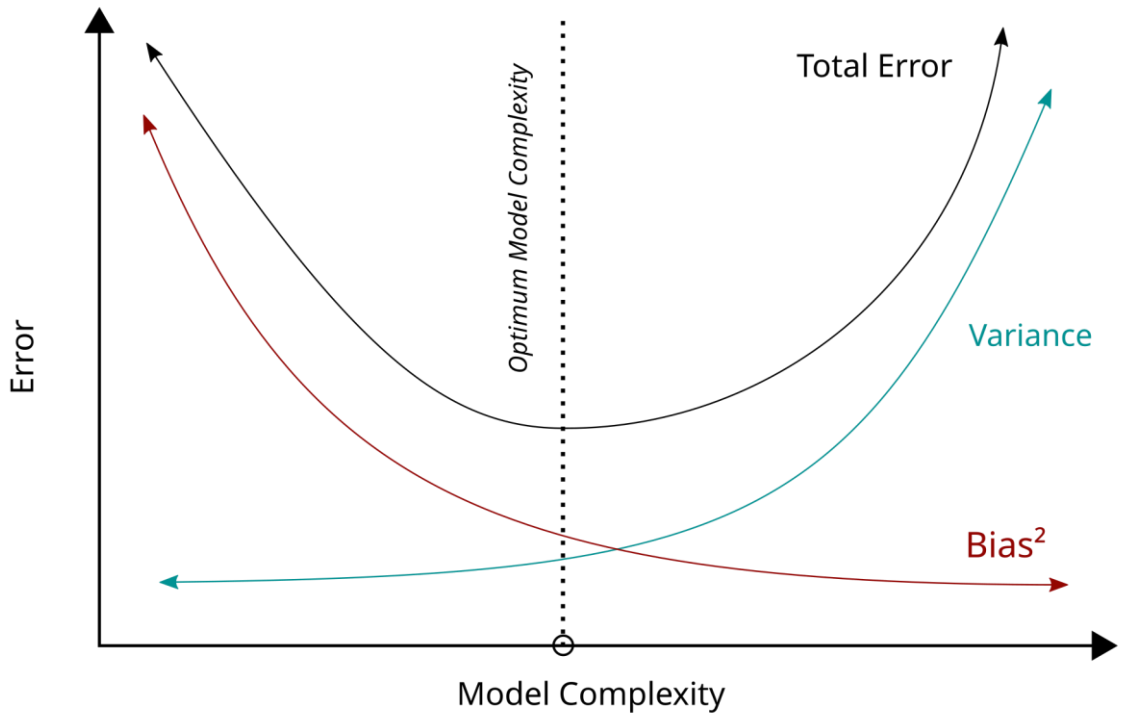


Figure courtesy: https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff#/media/File:Bias_and_variance_contributing_to_total_error.svg

Example - Marketing

“Using underfitted models for decision-making could be costly for businesses. For example, an underfitted model may suggest that you can always make better sales by spending more on marketing when in fact the model fails to capture a saturation effect (at some point, sales will flatten out no matter how much more you spend on marketing). If your business is relying on that model to determine your marketing budget, you will overspend on marketing.” (~DataRobot)

Possible solution

- Augmentation
- Regularization (Dropout)
- Early stopping
- Model complexity tradoff

MLP: A (dvantage) **and D (isadvantage)**

A Universal approximator;

A Provides posteriors in output;

A Numerous optimizer available.

D Training computationally demanding.

Slides adapted from:

- Pattern Recognition and Machine Learning Book by Christopher Bishop
- Kevin Murphy, UBC Machine learning course
- Mubarak Shah, UCF Computer Vision course
- Farid Melgani, UoT Pattern Recognition course
- Pattern Classification Book by Duda and Hart